

RAPTORENGINE

Build, Development & Runtime Platform

Technical Whitepaper v0.1

RaptorEngine is the semantic build and runtime platform for RaptorJS: compiler-first, graph-aware, state-preserving and designed to optimize the complete path from source code to browser, server and RaptorWire.

Concept / architecture draft - September 2026

Status: design target. Performance claims in this document are objectives until validated by implementation benchmarks.

Executive summary

RaptorEngine este propus ca platforma proprie de development, build si server runtime pentru ecosistemul Raptor. Scopul sau nu este sa rescrie de dragul originalitatii fiecare componenta a toolchain-ului JavaScript, ci sa detina acele straturi in care cunoasterea semantica a RaptorJS, RaptorWire si RaptorTest poate produce optimizari pe care un bundler generic nu le poate face.

Arhitectura propusa separa doua niveluri. La nivelul low-level, versiunile initiale pot folosi Rolldown pentru bundling si Oxc pentru parsing, transformari si minificare. La nivelul semantic, RaptorEngine construiește propriul Raptor IR si un Semantic Application Graph care reprezinta componente, signals, derived values, DOM bindings, server queries, RaptorWire schemas, Reactive Address Space, routes si frontiere client/server.

Aceasta separare permite o evolutie pragmatica: primul release poate oferi un CLI unificat si optimizari Raptor fara a investi ani intr-un bundler general-purpose. Daca benchmark-urile demonstreaza ulterior ca anumite componente low-level limiteaza performanta sau functionalitatea, ele pot fi inlocuite incremental.

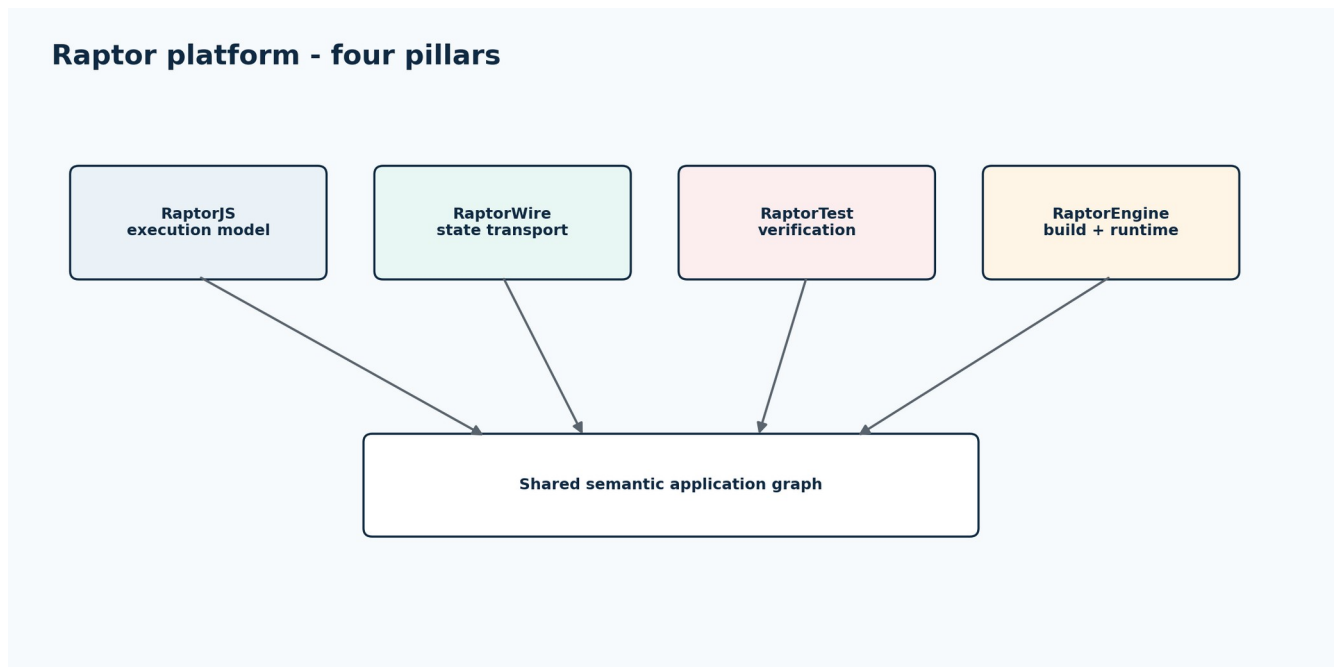


Figura 1. RaptorEngine devine al patrulea pilon al platformei Raptor.

Document map

Zona	Capitole
Fundatie	1-4: viziune, principii, pozitionare si arhitectura
Compiler	5-9: pipeline, IR, graph model si code generation
Development	10-12: incremental engine, Reactive HMR si diagnostics
Build	13-18: optimizari, chunking, CSS/assets, wire co-optimization si profiles
Runtime	19-21: server runtime, deployment si caching
Inteligenta	22-24: profiling, RaptorTest telemetry si behavior-guided build
Platforma	25-31: plugins, security, compatibility, benchmarks, MVP, roadmap
Specificatie	Appendix: config, CLI, IR examples, cache manifest si referinte

1. Viziune si obiective

RaptorEngine nu este definit ca un simplu bundler. El este stratul care intelege sensul aplicatiei Raptor si transforma aceasta cunoastere in compilare incrementală, optimizare, HMR, output browser/server/wire si observabilitate.

1.1 Obiective de produs

- Un singur CLI pentru development, build, run, test, profile, inspect si analyze.
- Compilare semantic-aware: optimizarile pot traversa componenta, reactive graph-ul, server queries si protocolul RaptorWire.
- Feedback loop foarte rapid prin invalidare minimala si cache semantic persistent.
- Stateful Reactive HMR: schimbarile de cod sa pastreze cat mai mult din starea browserului si a sesiunii RaptorWire.
- Build reproductibil, inspectabil si usor de diagnosticat.
- Abilitatea de a folosi tool-uri low-level excelente fara a expune dependenta lor in DX-ul Raptor.

1.2 Non-obiective initiale

- Nu rescriem imediat un parser JavaScript complet.
- Nu construim un minifier general-purpose inainte ca RaptorJS sa aiba workload-uri reale.
- Nu urmarim compatibilitate cu fiecare plugin Webpack/Vite in v0.1.

- Nu promitem superioritate de performanta pana cand benchmark-urile sunt publice si reproductibile.

Principiu: own the semantics first, own the low-level primitives only when measurements justify it.

2. Contextul toolchain-ului in 2026

Piata JavaScript a evoluat deja puternic spre toolchain-uri Rust si incremental computation. Vite 8 a trecut la Rolldown ca bundler unificat; Rolldown este conceput explicit ca motorul de bundling pentru Vite. Oxc ofera parser, transformer, resolver si minifier in Rust, iar Next.js/Turbopack investeste in caching incremental si compilare Rust. Bun ofera propriul build API si runtime integrat.

Sistem	Punct forte relevant	Lectie pentru RaptorEngine
Vite 8 + Rolldown	DX matur, HMR, bundling Rust unificat	Nu reinventam bundling-ul general-purpose in v0.1.
Oxc	Parser/transform/minify/resolver Rust de mare performanta	Poate servi ca baza low-level a compilerului.
Turbopack	Incremental computation si persistent caching	Raptor trebuie sa invalideze semantic, nu doar pe fisiere.
Bun	Runtime + build + tooling integrat	Un toolchain unificat poate imbunatati DX-ul.
RaptorEngine	Cunoastere end-to-end a reactive graph + wire + test telemetry	Diferentierea trebuie sa fie semantica, nu doar viteza bruta.

RaptorEngine trebuie deci sa concureze prin optimizari specifice modelului Raptor: Dead Signal Elimination, Dependency Fusion, compiled network bindings, schema generation, profile-guided reactive strategies si HMR la nivel de graph.

3. Principii de proiectare

Semantic before syntactic. Engine-ul optimizeaza sensul aplicatiei, nu doar token-uri si module.

Incremental by default. Orice editare invalideaza cea mai mica regiune corecta din graph.

State preservation. Development-ul nu trebuie sa distruga inutil starea utilizatorului sau a sesiunii.

Measure everything. Build-ul produce telemetry si explicatii pentru fiecare optimizare semnificativa.

Deterministic output. Aceeasi intrare + aceeasi configuratie produce acelasi output.

Progressive ownership. Rolldown/Oxc pot ramane primitive pana cand exista un motiv masurabil sa fie inlocuite.

Interop as migration path. Raptor poate consuma pachete NPM si poate coexista cu codebase-uri existente.

Performance budgets as tests. Regresiile de build/runtime sunt tratate ca erori de engineering, nu ca impresii.

4. Arhitectura de ansamblu

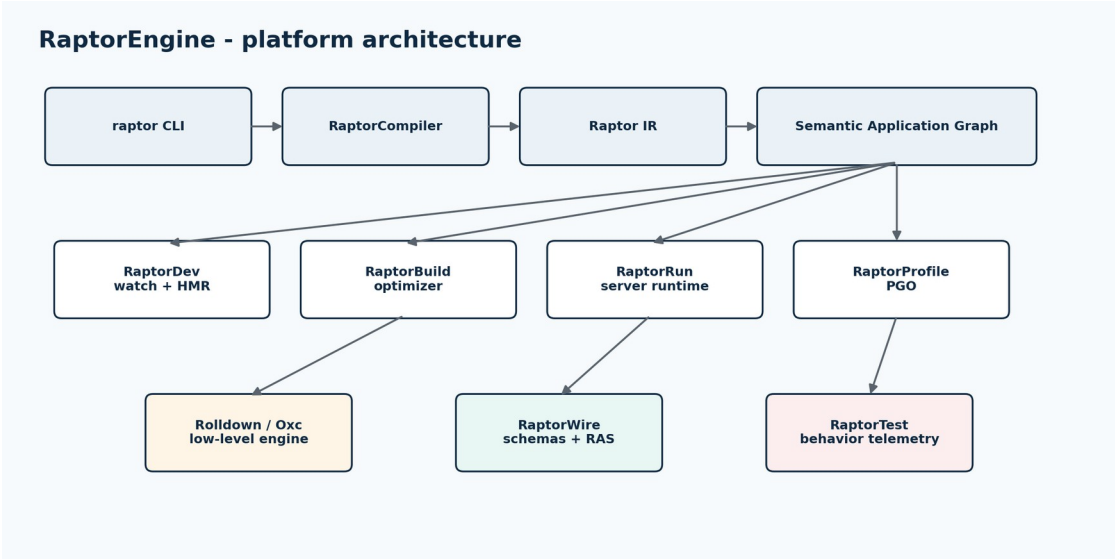


Figura 2. Arhitectura RaptorEngine si dependentele sale principale.

Semantic Application Graph este centrul arhitecturii. File graph si module graph raman importante, dar ele nu sunt suficiente pentru optimizarile Raptor. Engine-ul trebuie sa stie ce signal alimenteaza ce derived expression, ce binding DOM consuma rezultatul, ce query server produce datele si ce schema wire este necesara pentru transport.

Componenta	Responsabilitate
raptor CLI	Intrarea unica pentru developer si CI.
RaptorCompiler + IR	Parseaza, analizeaza si construieste reprezentarea intermediara Raptor.
Semantic Application Graph	Leaga module, signals, DOM, server, routes si wire.
RaptorDev + RaptorBuild	Watch, incremental compile, HMR, optimization si output deployment-ready.
RaptorRun	Runtime server pentru SSR, server functions, RaptorWire si routing.
RaptorProfile + RaptorTest	Profile pentru PGO si behavior telemetry pentru build-uri ghidate de utilizare.

5. Developer experience si CLI

Developerul nu trebuie sa cunoasca motorul de bundling intern. Contractul public este CLI-ul Raptor si configuratia Raptor.

```
npm create raptor@latest
cd my-app
raptor dev
raptor build
raptor test
raptor profile
raptor inspect
```

Comanda	RoI
raptor dev	Porneste dev server, watch graph, HMR si diagnostics.
raptor build	Construieste output browser/server/wire si manifestele.
raptor run	Ruleaza runtime-ul server din build sau in mod development.
raptor preview	Serveste build-ul local in conditii apropiate de productie.
raptor test	Delegare catre RaptorTest.
raptor profile	Colecteaza telemetry pentru profile-guided optimization.
raptor inspect	Vizualizeaza graph, IR, addresses, chunks si invalidari.
raptor analyze	Analizeaza bundle, signals, routes si costurile runtime.

6. Compiler pipeline

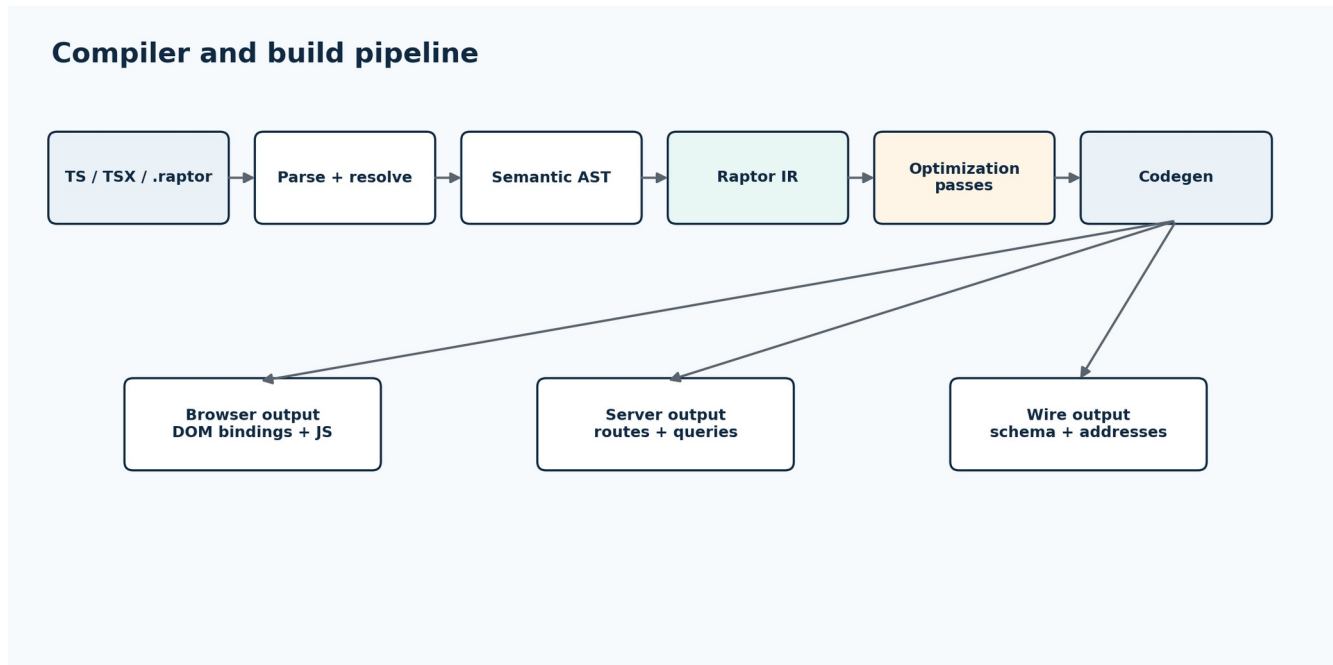


Figura 3. Pipeline-ul compilerului produce simultan artefacte pentru browser, server si wire.

RaptorCompiler trebuie sa fie separat de bundler. El poate fi consumat din CLI, IDE, test runner sau eventual din alte build systems. Aceasta separare face posibil ca @raptor/compiler sa ramana nucleul stabil in timp ce engine-ul low-level evolueaza.

1. Parse source cu Oxc sau parser echivalent.
2. Resolve imports si metadata de framework.
3. Construct Semantic AST.
4. Lower la Raptor IR.
5. Construiești reactive/server/wire graph.
6. Ruleaza optimization passes.
7. Genereaza browser/server/wire artifacts.
8. Trimite modulele JavaScript finale catre bundler/minifier.

7. Raptor IR

Raptor IR este mecanismul prin care framework-ul poate optimiza dincolo de JavaScript generic. Componenta nu mai este doar o functie; este descompusa in noduri, bindings, dependencies, effects, queries si transitions.

```

source:
  const count = state(0)
  <button on:click={() => count++}>Count: {count}</button>

```

```

IR (conceptual):
  signal s1 : int = 0
  node n1 : button

```

```
event e1 : click -> inc s1 +1
text_binding b1 : "Count: " + s1 -> n1.text
```

7.1 Proprietati dorite

- Typed acolo unde tipurile sunt cunoscute.
- SSA-like pentru expresii pure unde este util.
- Stable IDs pentru graph diff si HMR.
- Source maps inapoi la TS/TSX/.raptor.
- Serializable partial pentru caching si inspect tooling.
- Extensibil pentru server queries si RaptorWire schema nodes.

8. Semantic Application Graph

Graful semantic leaga mai multe tipuri de dependente intr-o singura vedere coerenta.

```
Module -> Component -> Signal -> Derived -> DOM Binding
                                     \-> ServerQuery -> WireSchema -> RAS Address
Route  -> Layout -> Component
TestFlow -> Route/Action -> Runtime profile
```

Aceasta reprezentare permite raspunsuri pe care un module graph normal nu le poate oferi: daca un signal se modifica in compilare, care bindings trebuie regenerate? Daca o schema wire se schimba, care clienti sunt incompatibili? Daca un route nu foloseste niciodata un derived value, poate fi eliminat din chunk-ul sau?

9. Code generation multi-target

Target	Output
Browser	JS minimal, DOM bindings, reactive slots, event handlers, hydration/resume metadata daca este necesar.
Server	Server functions, route handlers, SSR/resume output, query/subscription code.
RaptorWire	Schema IDs, codecs, Reactive Address Space manifest, compatibility metadata.
DevTools	Symbol maps intre source, IR, signals, addresses si DOM bindings.
Test	Optional instrumentation pentru RaptorTest si deterministic replay.

Un obiectiv cheie este ca aceste outputs sa fie generate din acelasi graph semantic, reducand drift-ul dintre client, server si protocol.

10. RaptorDev: incremental engine

RaptorDev trebuie sa optimizeze timpul edit -> feedback, nu doar viteza unui build complet. File watching-ul este doar primul nivel; invalidarea reala trebuie sa fie dependency-aware si semantic-aware.

Schimbare	Invalidare ideala
CSS local intr-o componenta	Doar CSS output si style binding.
Text static	Template fragment / DOM graph local.
Expresie derived	Derived node + dependente downstream.
Tip shared folosit de wire	Schema analysis + client/server codecs dependente.
Route metadata	Route manifest si chunk planning, nu intregul app.
Server query	Server code + wire mapping + bindings care depind de rezultat.

Incremental graph and cache model

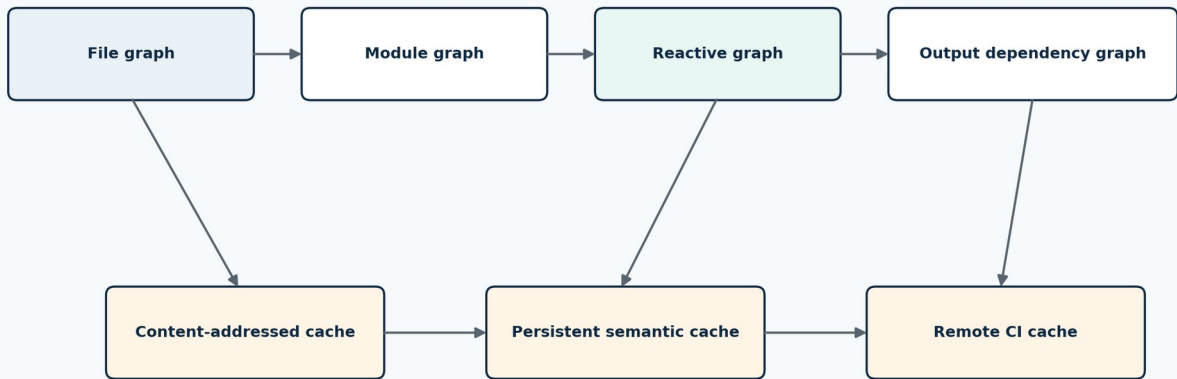


Figura 4. Engine-ul incremental foloseste mai multe grafuri si cache-uri corelate.

11. Stateful Reactive HMR

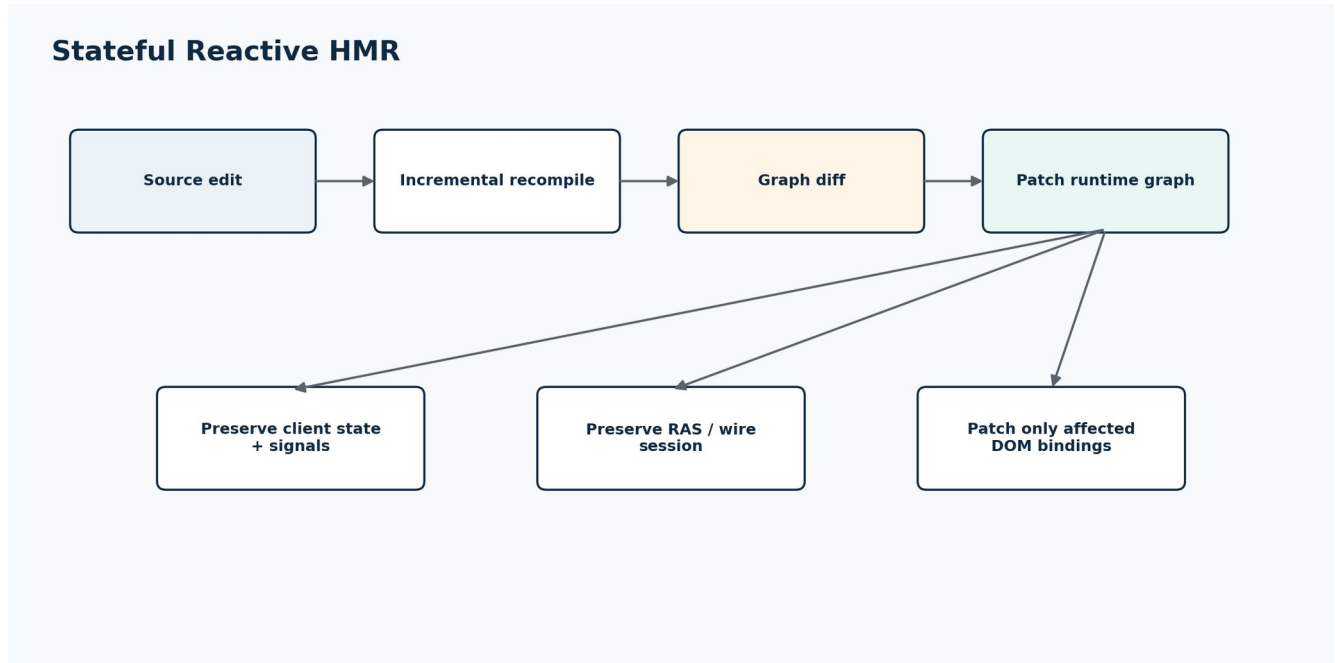


Figura 5. HMR-ul propus modifica graful compilat pastrand starea compatibila.

HMR-ul clasic inlocuieste module si incearca sa pastreze starea framework-ului. Raptor poate merge mai jos: folosind stabile IDs din IR, engine-ul poate compara graph-ul vechi cu cel nou si poate patch-ui numai nodurile modificate.

11.1 Stare candidata pentru preservare

- Local signals compatibile ca tip si identitate.
- Store-uri si cache-uri remote.
- RaptorWire session si address mappings.
- Route location si scroll state.
- Form state unde structura ramane compatibila.
- RaptorTwin state in development test mode.

11.2 Fallback sigur

Daca modificarea schimba incompatibil contractul unui signal, schema sau component boundary, engine-ul trebuie sa faca fallback explicit la remount/reload si sa explice motivul in overlay. Predictibilitatea este mai importanta decat preservarea agresiva.

12. Diagnostics si explainability in development

RaptorDev trebuie sa explice ce face. Un build foarte rapid, dar opac, devine dificil de depanat cand invalidarea sau optimizarea este gresita.

```
RaptorDev update #184
changed: src/PriceCard.raptor
invalidated: component PriceCard
preserved: 12 signals, 1 wire session
regenerated: 2 DOM bindings, 0 routes, 0 schemas
graph patch: 1.8 ms
browser apply: 0.6 ms
```

raptor inspect poate afisa graph-ul, dependentele unui signal, motivul includerii unui modul intr-un chunk si provenance-ul unui RAS address.

13. RaptorBuild: optimization pipeline

RaptorBuild ruleaza optimizari generale prin engine-ul low-level si optimizari semantic-specific prin Raptor IR. Ordinea este importanta pentru a nu pierde informatie de framework prea devreme.

Faza	Exemple
Semantic pruning	Dead Signal Elimination, unused binding removal, unused server-query branch removal.
Reactive optimization	Dependency Fusion, effect collapsing, batching hints, static dependency extraction.
Cross-boundary optimization	Compiled network bindings, schema specialization, server/client placement.
Module optimization	Tree shaking, side-effect analysis, chunking.
Syntax/minify	Lowering, compression, mangling.
Manifest generation	Routes, chunks, RAS, wire schemas, preload graph, diagnostics.

14. Dead Signal Elimination si Dependency Fusion

14.1 Dead Signal Elimination

```
const x = state(10)
const y = derived(() => x * 2)
const z = derived(() => y + 10) // never consumed
```

=> eliminate z and its subscription metadata

Eliminarea nu se limiteaza la variabile nefolosite textual; se face pe reactive graph. Un signal sau derived poate fi eliminat daca nu are consumatori observabili si nu produce side effects.

14.2 Dependency Fusion

```
a -> derived b -> derived c -> DOM

if b and c have no other consumers:

a -> fused expression -> DOM
```

Fusion poate reduce closures, subscriptions, scheduler entries si allocations. Trebuie insa blocata cand exista debugging boundaries, observability hooks sau semantics care ar fi alterate.

15. Compiled network bindings si co-optimizarea RaptorWire

Pentru server signals si live queries, build-ul cunoaste simultan producer-ul server-side, schema wire si consumer-ul browser-side. Acest lucru permite generarea unui traseu compact.

```
serverSignal("BTC.price")
-> RAS 0x18A1
-> wire decoder slot 7
-> client reactive slot 3
-> DOM binding 22
```

RaptorBuild poate genera schema IDs, codecs specializate si mapping-uri precompilate. Daca tipul este cunoscut si range-ul este declarat, encoding-ul poate evita reprezentari generale. Aceste optimizari raman conditionate de benchmark-uri si de costul de compatibilitate.

16. Chunking semantic si route-aware

Chunking-ul nu trebuie ghidat exclusiv de module imports. Semantic graph-ul poate cunoaste routes, layouts, probabilitati de navigare din profile si grupuri de signals/queries folosite impreuna.

Semnal semantic	Posibila decizie
Module mereu folosite impreuna	Fold intr-un singur chunk pentru mai putine request-uri.
Feature rar accesat	Lazy chunk dedicat.
Route critic initial	Preload mai agresiv.
Heavy realtime decoder	Worker chunk separat.
Shared server/client schema	Genereaza codecs target-specific fara duplicarea metadatelor inutile.

17. CSS, assets si static resources

RaptorEngine trebuie sa trateze CSS si assets pragmatic. v0.1 poate delega transformarea de baza catre toolchain-ul existent, dar graph-ul trebuie sa cunoasca ownership-ul componentelor si routes pentru preload, extraction si invalidare.

- Scoped CSS optional la nivel de componenta.
- Critical CSS extraction per route in full-stack mode.
- Asset hashing content-addressed.
- Image/font metadata pentru preload fara a obliga un image pipeline proprietar in MVP.
- CSS HMR fara remount componenta.

18. Build profiles

RaptorBuild poate avea profile explicite pentru workload-uri cu caracteristici diferite. Profilele nu trebuie sa ascunda magie; ele sunt preseturi inspectabile de strategii.

Profil	Optimizari prioritare
default	Echilibru startup/build/runtime.
realtime	Decoder rapid, batch scheduling mic, worker-friendly wire processing.
dashboard	Grafuri reactive mari, batching DOM, list virtualization hints.
content	SSR/static output, JS minim, route splitting, SEO.
edge	Server bundle mic, cold start si API compatibility.
embedded	Output compact, feature stripping agresiv.

```
raptor build --profile realtime
```

19. RaptorRun: server runtime

Browserul va continua sa ruleze pe motorul JavaScript al browserului. "Runtime propriu" inseamna server runtime si platforma de executie Raptor, nu inlocuirea V8/SpiderMonkey/WebKit in client.

19.1 Responsabilitati

- Routing server si request lifecycle.
- SSR/resume orchestration.
- Server functions si server signals.
- RaptorWire connections si session state.
- Adapters DB/cache/queues.
- Static assets si manifests.
- Background tasks limitate si hooks de platforma.
- Observability si graceful shutdown.

19.2 Runtime targets

Target	Strategie
Node.js	Primul target de compatibilitate, ecosistem maxim.
Bun	Optional target pentru startup/runtime diferit.
Edge runtimes	Subset de API-uri si adapters.
Standalone binary	Posibil ulterior, daca toolchain-ul ales permite packaging robust.

20. Routing si full-stack execution

RaptorEngine poate deveni baza unui RaptorKit/RaptorStart ulterior, dar routing-ul minim trebuie modelat devreme pentru chunking si server output.

```
routes/  
  index.raptor  
  products/[id].raptor  
  admin/layout.raptor
```

```
Build outputs:  
  route manifest  
  browser chunks  
  server handlers  
  preload graph  
  wire/query metadata
```

21. Caching, reproducibility si CI

Caching-ul este esential pentru un engine incremental. Un cache semantic poate reutiliza nu doar output-ul unui fisier, ci parse trees, IR fragments, schema analysis si graph edges cand input-urile relevante nu s-au schimbat.

21.1 Cache keys

- Content hash al source-ului.
- Compiler version si optimization flags.
- Resolved dependency signatures.
- Relevant environment/define values.
- Target + build profile.
- Schema compatibility version.

21.2 Reproducibility

Build-ul trebuie sa poata emite un build manifest cu versiunile toolchain-ului, hashes, flags si input provenance. Timestamp-urile non-deterministe nu trebuie sa intre in artifact hashes implicit.

22. RaptorProfile: runtime telemetry pentru compiler

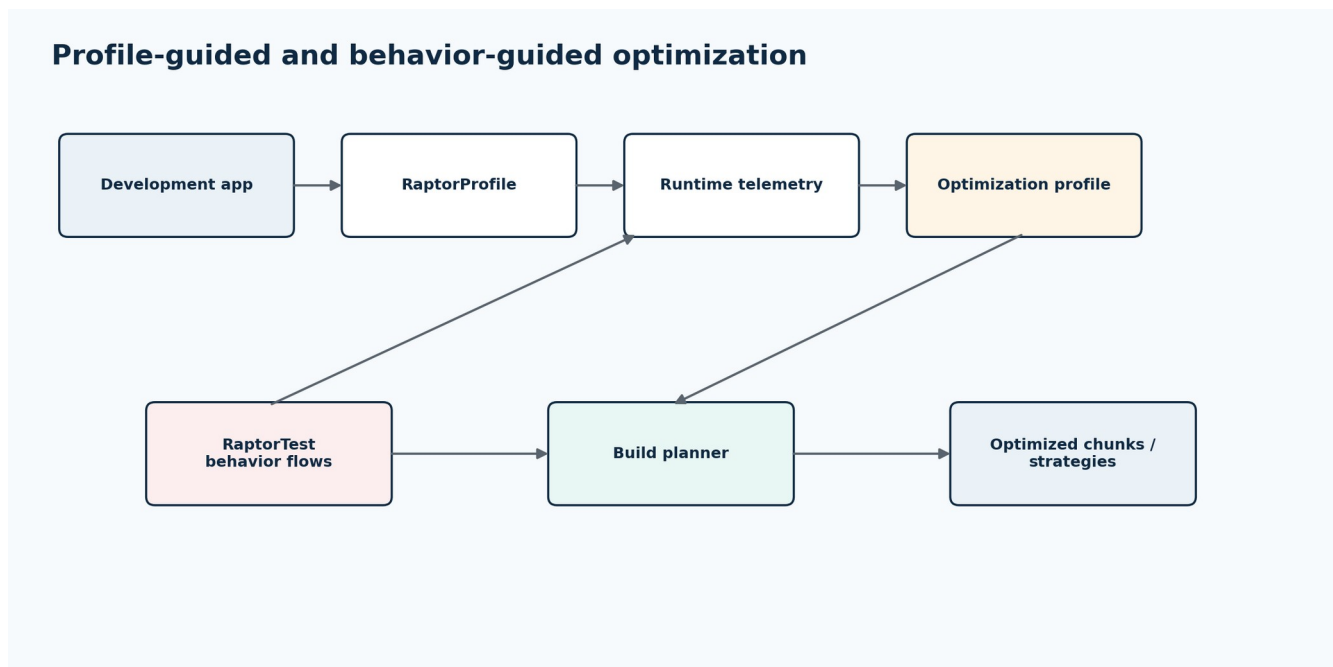


Figura 6. Profile-guided optimization combina telemetry runtime cu behavior flows din RaptorTest.

Profiling-ul trebuie sa fie optional si controlat. Scopul sau este sa masoare comportamente tehnice, nu sa colecteze date personale ale utilizatorilor.

Metric	Utilizare potentiala
Signal update frequency	Alege scheduling/inline strategy.
Derived fan-out	Identifica hot reactive nodes.
Route frequency	Chunk/preload planning.
Chunk co-usage	Chunk folding/splitting.
Wire payload distribution	Encoding specialization.
DOM mutation bursts	Batch sizing.
Cold vs hot paths	Code placement si lazy loading.

23. Behavior-guided builds cu RaptorTest

RaptorTest poate furniza flows reprezentative: ce routes sunt parcurse impreuna, ce actiuni declanseaza anumite queries, ce states apar in practica. Aceste date pot imbunatati build planning-ul fara a face optimizarile dependente de un singur traseu observat.

```
RaptorTest flows
-> normalized behavior profile
-> RaptorBuild planner
-> chunk/preload candidates
-> safety constraints
-> optimized build
```

Behavior-guided optimization trebuie sa fie hint-based. Correctness-ul build-ului nu poate depinde de faptul ca un flow a fost observat in profil.

24. Adaptive strategies, nu adaptive correctness

Profilele pot influenta strategia, nu semantica. De exemplu, doua signals pot fi fused doar daca analiza statica dovedeste echivalenta. Faptul ca un signal nu a fost vazut activ intr-un profil nu este suficient pentru a-l elimina.

Permis	Interzis
Ajustare chunk boundaries pe baza co-usage	Eliminarea codului doar fiindca nu a aparut in profile.
Alegerea unui batch size diferit	Schimbarea ordinii semantice a effects.
Preload pentru routes frecvente	Ignorarea routes rare din output.
Specializare wire cu fallback compatibil	Encoding incompatibil fara negotiation/versioning.

25. Plugin model si extensibilitate

RaptorEngine trebuie sa ofere extensibilitate, dar sa evite ca fiecare plugin sa poata muta arbitrar orice faza a compilerului. Stabilitatea IR-ului si a semantics este prioritara.

25.1 Clase de plugins

Tip	Exemple de hook-uri
Source loader	Extensii de fisiere, virtual modules.
AST/IR transform	Transformari validate la faze documentate.
Asset	Load/transform assets.
Server adapter	Deployment/runtime adapters.
Analyzer	Read-only graph/IR analytics.
Dev overlay	Diagnostics custom, fara mutarea semantics.
Build output	Manifest consumers si post-processing controlat.

Compatibilitatea cu plugin-uri Rollup/Vite poate exista prin adapter pentru use cases simple, dar nu trebuie promisa 1:1 daca semantics nu pot fi garantate.

26. Security si supply chain

- Plugin permissions sau cel putin provenance clar pentru plugins care ruleaza cod la build.
- Lockfile awareness si reproducible dependency resolution.
- Separarea stricta a variabilelor publice de server secrets.
- Build manifest cu dependency/toolchain versions.
- Optional SBOM output.
- Nu se include telemetry de profil in artifact daca nu este necesar.
- Server functions trebuie validate pentru boundary leaks si accidental client bundling.

26.1 Secret boundary analysis

Semantic graph poate detecta tentative de a transporta valori marcate server-only catre browser. Acest tip de diagnostic este un avantaj al unui engine framework-aware.

27. Observability si DevTools

RaptorEngine trebuie sa produca date pentru RaptorDevTools fara a livra metadata voluminoasa in productie implicit.

Vizualizare	Intrebare rezolvata
Module graph	De ce este acest pachet in build?
Reactive graph	Ce depinde de signal-ul X?
RAS map	Ce reprezinta address 0x18A1?
Chunk graph	Ce route incarca acest chunk?
Invalidation trace	De ce editarea unui fisier a recompilat alte 17 noduri?
Optimization trace	Ce a fost eliminat/fused si de ce?
Wire schema diff	Ce incompatibilitate a aparut intre doua build-uri?

28. Interoperabilitate si migration path

Adoptia Raptor depinde de abilitatea de a folosi ecosistemul NPM si de a migra incremental. Engine-ul trebuie sa rezolve ESM/CJS compatibil, package exports, CSS imports si common asset patterns.

- Import standard din npm.
- Web Components ca boundary de interoperabilitate.
- Raptor Islands in aplicatii non-Raptor, ulterior.
- Adapters pentru server deployments comune.
- Optional build via Vite adapter pentru debugging/migration, fara a face Vite dependenta arhitecturala.

29. Benchmark methodology

Benchmark-urile trebuie separate pe build-time, dev-time si runtime. Un scor agregat unic ascunde trade-off-uri.

Categorie	Masuratori
Cold build	wall time, CPU, peak RSS, output size.
Warm incremental build	single-file edit latency, graph invalidation size.
HMR	edit -> browser applied latency; preserved state ratio.
Large repo	scaling cu module count si parallelism.
Runtime browser	startup JS, allocations, signal update cost, DOM mutations.
Wire	encode/decode throughput, bytes/update, resync cost.
Server runtime	cold start, requests/sec, memory, RaptorWire session cost.
CI cache	cache hit ratio, transfer size, rebuild after branch changes.

29.1 Comparatoare

Pentru build/dev: Vite 8/Rolldown, Bun build si scenariii relevante Next/Turbopack. Pentru runtime UI: benchmark-urile RaptorJS trebuie separate si comparate cu Solid, Svelte, React si Qwik. Comparatiile trebuie publicate cu hardware, versions, source si raw results.

30. Performance budgets propuse

Buget	Tinta initiala / regula
No-op incremental edit	Sa nu regenereze outputs neafectate.
Graph invalidation	Proportionala cu dependentele reale, nu cu proiectul complet.
Dev startup	Masurat pe repo-uri small/medium/large; regression gate.
HMR state preservation	Maximizat pentru schimbari structural compatibile.
Build determinism	Identical inputs => identical content hashes.
Raptor metadata production	Strip by default; debug/profile artifacts separate.
Runtime overhead	Orice abstraction noua trebuie justificata prin benchmark.

Nu sunt rezultate demonstrate; sunt criterii de acceptanta pentru implementare.

31. MVP RaptorEngine v0.1

MVP-ul trebuie sa demonstreze semantic compilation si feedback loop, nu sa egaleze functional fiecare build system matur.

Milestone	Scope
M1 CLI skeleton	raptor dev/build/inspect; config loading; project discovery.
M2 Compiler integration	TS/TSX parse, minimal Raptor syntax, IR generation.
M3 Graph core	module graph + reactive graph + stable IDs.
M4 Dev server	watch, incremental compile, browser reload/HMR baseline.
M5 Reactive HMR	graph diff si preservation pentru signals simple.
M6 Production build	Rolldown/Oxc integration, code splitting, manifests.
M7 Wire integration	schema/RAS manifest generation.
M8 Inspect/analyze	graph dump, invalidation trace, build metadata.
M9 Benchmark suite	cold/warm/HMR/runtime integration benchmarks.

31.1 Definition of done pentru v0.1

- Poate construi si servi o aplicatie RaptorJS reala.
- Dev loop nu depinde de Vite ca API public.
- Build production foloseste engine low-level prin adapter intern.
- Compiler si IR sunt independente de bundler.
- Exista benchmark-uri reproducibile si regression gates.

32. Roadmap spre v1.0

Versiune	Directie
v0.1	CLI + compiler integration + graph + Rolldown build + basic HMR.
v0.2	Stateful Reactive HMR, RAS/wire manifests, richer diagnostics.
v0.3	Server runtime baseline, SSR/resume primitives, deployment adapters.
v0.4	Semantic optimizer: DSE, dependency fusion, network bindings.
v0.5	RaptorProfile + PGO experiments.
v0.6	RaptorTest behavior profiles + build planner.
v0.7	Remote cache, monorepo scaling, CI optimization.
v0.8	Plugin API stabilizare, compatibility tooling.
v0.9	Hardening, security, observability, ecosystem validation.
v1.0	Stable compiler/IR contracts, supported targets, published benchmarks.

33. Structura propusa a repository-ului

```
raptor/  
  packages/  
    compiler/  
      ir/  
      graph/  
      cli/  
      dev/  
      build/  
      run/  
      profile/  
      wire-codegen/  
      devtools-protocol/  
      adapters/  
  crates/  
    raptor_parser_bridge/  
    raptor_graph/
```



```

raptor_ir/
raptor_optimizer/
raptor_cache/
raptor_dev_server/
benchmarks/
examples/
docs/

```

Limba principala pentru engine-ul de performanta este propusa a fi Rust, cu bindings N-API/WASM unde sunt necesare. API-urile consumate de developer raman TypeScript-friendly.

34. Configuratie propusa

```

import { defineConfig } from "@raptor/engine"

export default defineConfig({
  target: "web",
  profile: "default",
  server: { runtime: "node" },
  build: {
    sourcemap: true,
    report: true
  },
  wire: {
    enabled: true,
    schemaMode: "strict"
  },
  dev: {
    statefulHmr: true
  }
})

```

Configuratia trebuie sa aiba defaults bune. Fisierele separate de configurare sunt justificate doar pentru optiuni care nu pot fi inferate din proiect.

35. Exemplu end-to-end de compilare

```

Source:
const price = serverSignal<number>("BTC.price")
const formatted = derived(() => price.toFixed(2))

<span class="price">${formatted}</span>

Semantic graph:
server producer -> schema(decimal2) -> RAS 0x18A1
0x18A1 -> client signal s1 -> derived d1 -> DOM binding b1

Optimization candidate:
if d1 has one consumer and debug boundary allows fusion:
0x18A1 -> decode -> format -> b1

Outputs:
server: producer registration
wire: schema + address manifest
browser: decoder slot + binding update function

```

Exemplul arata motivul existentei RaptorEngine: aceeași informație traversează server, protocol și UI. Un bundler generic poate optimiza modulele, dar nu cunoaște în mod natural acest traseu semantic complet.

36. Riscuri tehnice

Risc	Mitigare
Complexitate prea mare prea devreme	MVP restrans și folosirea Rolldown/Oxc ca primitive.
IR devine instabil	Versionare internă, golden tests, documented invariants.
HMR preservation produce bug-uri greu de înțeles	Compatibility checks + explicit fallback.
Plugin ecosystem fragmentat	API mic, faze clare, adapters pentru tooling existent.
PGO overfits profile	Hints only; static correctness independent.
Compiler optimizations schimbă semantics	Differential tests, interpreter/reference mode.
Toolchain Rust mare pentru contributori JS	N-API wrappers, TypeScript APIs, dev docs și prebuilt binaries.
Build speed devine obiectiv în detrimentul runtime-ului	Separate scorecards și budgets.

37. Criterii pentru a înlocui Rolldown/Oxc

RaptorEngine nu trebuie să își scrie propriul bundler/parser din orgoliu. O componentă low-level se înlocuiește doar când există un caz demonstrat.

- Un optimization pass Raptor important nu poate fi exprimat eficient prin API-ul existent.
- Profiling-ul arată ca acea componentă domina semnificativ latency/memory.
- Integrarea impune transformări redundante sau pierdere de metadata semantică.
- Costul de mentenanță al unui fork/engine propriu este mai mic decât costul limitărilor.
- Există benchmark și test suite suficiente pentru a valida replacement-ul.

Originalitatea RaptorEngine trebuie să fie măsurată prin capacitatea sa de a elimina munca inutilă end-to-end, nu prin numărul de componente rescrise.

38. Criterii de succes ale produsului

Dimensiune	Semn de succes
Developer UX	Un developer poate porni, construi, inspecta si testa proiectul fara configurare de bundler.
Feedback loop	Editari locale produc invalidari locale si HMR previzibil.
Runtime	Optimizari semantice reduc efectiv work-ul din browser/server/wire.
Diagnostics	Developerul poate explica de ce un modul sau binding exista in output.
CI	Build-uri reproductibile, cacheabile si usor de comparat.
Ecosystem	Pachete npm uzuale functioneaza fara friction excesiv.
Credibilitate	Benchmark-urile sunt publice, reproductibile si nu confunda obiectivele cu rezultatele.

39. Concluzie

RaptorEngine transforma Raptor dintr-un framework frontend cu protocol propriu intr-o platforma de programare coerenta. Valoarea nu vine din faptul ca exista o comanda diferita de vite build, ci din faptul ca engine-ul cunoaste acelasi graph semantic pe care il folosesc RaptorJS, RaptorWire si RaptorTest.

Strategia recomandata este pragmatica: construim propriul compiler, IR, graph, dev loop, HMR, optimizer si runtime orchestration, dar folosim Rolldown/Oxc ca fundatie low-level in primele versiuni. Inlocuim ulterior doar componentele pentru care avem dovada ca o implementare proprie aduce un avantaj real.

RaptorJS = execution. RaptorWire = communication. RaptorTest = verification.
RaptorEngine = compilation, development and runtime orchestration.

Appendix A. CLI draft

```
raptor dev [root]
  --host
  --port <n>
  --profile <name>
  --inspect

raptor build [root]
  --profile <name>
  --target <web|server|edge>
  --report
  --profile-data <file>

raptor run [build]
  --port <n>

raptor inspect
  graph [module|reactive|route|wire]
  signal <id>
  address <hex>
  chunk <name>

raptor profile
  --scenario <file>
  --out raptor.profile

raptor analyze
  --bundle
  --reactivity
  --wire
```

Appendix B. Build manifest draft

```
{
  "engineVersion": "0.1.0",
  "compilerVersion": "0.1.0",
  "profile": "default",
  "inputsHash": "...",
  "toolchain": {
    "rolldown": "...",
    "oxc": "..."
  },
  "outputs": [...],
  "routes": [...],
  "wireSchemas": [...],
  "rasManifest": "ras.manifest",
  "reproducible": true
}
```

Appendix C. Raptor IR draft node taxonomy

Categorie	Noduri conceptuale
Values	Const, Param, Local, Import, ServerValue.
Reactive	Signal, Derived, Effect, Resource, StoreSlot.
UI	Element, TextBinding, AttrBinding, EventBinding, ListBinding, Conditional.
Server	ServerFunction, ServerQuery, ServerSignal, RouteHandler.
Wire	Schema, Field, Codec, Address, Subscription, Transaction.
Control	Branch, Loop, Batch, TransactionBoundary.
Metadata	SourceSpan, DebugName, StableId, TargetConstraint.

Appendix D. Referinte tehnice si context de piata

Sursa	URL	Relevanta
Vite 8 announcement	https://vite.dev/blog/announcing-vite8	Vite 8 foloseste Rolldown ca bundler Rust unificat.
Rolldown Introduction	https://rolldown.rs/guide/introduction	Pozitionarea Rolldown ca bundler high-performance si motor pentru Vite.
Rolldown Bundler API	https://rolldown.rs/apis/bundler-api	API-uri programatice rolldown/watch/build.
Oxc - What is Oxc?	https://oxc.rs/docs/guide/what-is-oxc	Toolchain Rust pentru parser, transformer, resolver, minifier si alte primitive.
Oxc Parser	https://oxc.rs/docs/guide/usage/parser	Parser JavaScript/TypeScript in Rust.
Oxc Transformer	https://oxc.rs/docs/guide/usage/transformer	Pipeline de transformare TS/JSX si lowering.
Bun BuildConfig	https://bun.sh/reference/bun/BuildConfig	Exemplu de runtime si build tooling integrat.
Next.js/Turbopack updates	https://nextjs.org/blog	Context pentru incremental compilation, caching si toolchain Rust.

Nota: documentul descrie arhitectura propusa RaptorEngine. Referintele de mai sus sustin doar contextul toolchain-ului existent; ele nu reprezinta benchmark-uri RaptorEngine.